PROJECT REPORT

ORANGE PROJECT

DONE

Malzahar

Author(s):

Famura

Contents

1. Introduction ... 3
2. Project Description ... 3
3. Project Technologies ... 3
   3.1. Module(s) ... 3
   3.2. Coding Language(s) ... 4
   3.3. Libraries & External Additions ... 4
4. Project Architecture ... 4
   4.1. High-Level Overview ... 4
   4.2. Security Model ... 5
      4.2.1. Request Confidentiality (Client → Server control payload) ... 5
      4.2.2. Request Integrity (Client → Server) ... 6
      4.2.3. Response Authenticity + Confidentiality (Server → Client) ... 6
      4.2.4. Bulk Data Encryption (ZIP chunks) ... 6
   4.3. Request / Response Flow ... 7
      4.3.1. General Flow: ... 7
      4.3.2. Response Flow: ... 7
   4.4. Command-Line Interface (CLI) ... 7
   4.5. Folder Mapping ... 8
   4.6. ZIP Handling ... 8
   4.7. Chunked Transfer (Push / Pull) ... 9
      4.7.1. Push: ... 9
      4.7.2. Pull: ... 10
5. Project Analysis ... 10
   5.1. Test ... 10
   5.2. Upgradable Things ... 11
6. Project Comments ... 11
   6.1. Possible Usages ... 11
   6.2. To-Do's When Using ... 11
   6.3. Creator Comments ... 11
7. Credits ... 12
8. References ... 12
9. Disk Contents ... 12

1. Introduction

Malzahar is a special project made for understanding and creating a git-like infrastructure for simple files on daily uses. Rest of the report focuses on a more technical side while this introduction focuses more on a daily part of the project. The project born on a personal demand for using a push and pull mechanic for save files on LA Noire. I was using two separate computers for playing and neither Steam nor Rockstar Launcher did not automatically share save files. There was a way in Steam but I thought a system that is running in my web site(Nexus) would be a good project. So, I did add it to my system and now it is working flawlessly.

A note, this report only focuses on Malzahar, client side written in C++ for file transfer. For understanding the detailed structure of the whole system, visit the report of Nexus.

2. Project Description

Malzahar is a Windows-based secure file/folder synchronization and transfer client that communicates with a remote HTTPS server (configured to www.famura.site). The project provides a command-line tool that can:

- Authenticate users (login)
- Register a local folder to a remote scope (connect)
- Query remote metadata or availability (check)
- Download the remote folder state to the local machine (pull)
- Upload a local folder state to the server (push)
- Manage a specific server-side version of a folder (manage)

The transfer design focuses on:

- **Confidentiality:** Payload encryption (RSA + AES-GCM)
- **Integrity:** HMAC-SHA256 signing for request/response authenticity
- **Reliability for large data:** ZIP + **chunked** upload/download with retries

Malzahar is designed as an infrastructure tool for personal workflows (portfolio-grade utility) and can be integrated into other automation or deployment pipelines.

3. Project Technologies

3.1. Module(s)

- **Windows (Windows 11):**
  The entire project is designed to run in Windows systems. It uses Windows native APIs for networking, cryptography, file system operations, and process execution. Did not tested in lower Windows versions like Windows 7 or 10.

3.2. Coding Language(s)

- **C++ (C++14 Standard) – Visual Studio 2022:**
  The entire project is written in C++ and is set to compile with the C++ 14 standard with CMAKE. This provides modern language features and compatibility with Visual Studio 2026.

3.3. Libraries & External Additions

- **Windows / System:**
  - **WinHTTP (winhttp.h):** HTTPS requests for communication with the server.
  - **bcrypt (bcrypt.h):** AES-GCM encryption/decryption, random generation, HMAC (via SHA-256 HMAC provider).
  - **Crypt32 (wincrypt.h / crypt32.lib):** Base64 encode/decode, importing RSA public key from SubjectPublicKeyInfo (SPKI).
  - **std::filesystem:** Folder mapping, directory cleanup, ZIP temp files.

- **OpenSSL:**
  - **openssl/evp.h, openssl/pem.h, openssl/rsa.h:** RSA OAEP decryption for server responses (client-side private key usage) and compatibility handling for OAEP parameter variants.

- **Windows / Powershell:**
  - **Compress-Archive** to create ZIPs from folders.
  - **Expand-Archive** to extract ZIPs to a target folder.

4. Project Architecture

4.1. High-Level Overview

Malzahar is a **CLI client** that talks to a remote server through HTTPS endpoints.

Key endpoints used:

- /api/private/transfer (control channel: auth + commands)
- /api/private/transfer/pushzip (chunk upload endpoint)
- /api/private/transfer/zipchunk (chunk download endpoint)

The client supports two major transfer modes:

- **Push:** local folder → ZIP → AES-GCM chunk encryption → upload chunks
- **Pull:** request AES key + chunk plan → download encrypted chunks → AES-GCM decrypt → ZIP extract to folder

4.2. Security Model

Malzahar uses a layered model:

4.2.1. Request Confidentiality (Client → Server control payload)

- Builds a JSON request object containing:
  - request (login/connect/check/pull/push/manage)
  - name, password
  - data (scope/folder/version/key etc.)

```
std::string build_signed_request_payload(const std::string& request, const std::string& name,
    const std::string& password, const std::string& data) {
    std::string json = "{\"request\":\"" + request + "\",\"name\":\"" + name +
        "\",\"password\":\"" + password + "\",\"data\":\"" + data + "\"}";

    std::vector<unsigned char> spki = base64_decode(RSA_PUBLIC_KEY_B64);
    std::vector<unsigned char> encrypted = rsa_encrypt_oaep_spki(spki,
        std::vector<unsigned char>(json.begin(), json.end()));
    std::string encryptedB64 = base64_encode(encrypted);

    return build_signed_payload(encryptedB64);
}
```

- Encrypts this JSON with **RSA OAEP** using server's **public key** (SPKI base64 embedded in client).

```
    BCRYPT_OAEP_PADDING_INFO paddingInfo;
    paddingInfo.pszAlgId = BCRYPT_SHA256_ALGORITHM;
    paddingInfo.pbLabel = nullptr;
    paddingInfo.cbLabel = 0;

    DWORD outLen = 0;
    NTSTATUS status = BCryptEncrypt(hKey, (PUCHAR)data.data(), (ULONG)data.size(),
        &paddingInfo, nullptr, 0, nullptr, 0, &outLen, BCRYPT_PAD_OAEP);
    std::vector<unsigned char> out(outLen);
    status = BCryptEncrypt(hKey, (PUCHAR)data.data(), (ULONG)data.size(),
        &paddingInfo, nullptr, 0, out.data(), (ULONG)out.size(), &outLen, BCRYPT_PAD_OAEP);

    BCryptDestroyKey(hKey);
    LocalFree(pInfo);

    out.resize(outLen);
    return out;
}
```

- The encrypted blob is Base64-encoded.

```
std::string base64_encode(const std::vector<unsigned char>& data) {
    DWORD outLen = 0;
    CryptBinaryToStringA(data.data(), (DWORD)data.size(),
        CRYPT_STRING_BASE64 | CRYPT_STRING_NOCRLF, nullptr, &outLen);
    std::string out(outLen, '\0');
    CryptBinaryToStringA(data.data(), (DWORD)data.size(),
        CRYPT_STRING_BASE64 | CRYPT_STRING_NOCRLF, &out[0], &outLen);
    if (!out.empty() && out.back() == '\0') out.pop_back();
    return out;
}
```

4.2.2. Request Integrity (Client → Server)

- Computes **HMAC-SHA256** over the base64 encrypted blob.

```
std::string build_signed_payload(const std::string& encryptedBase64) {
    auto sig = hmac_sha256(g_hmac_key, encryptedBase64);
    std::string sigB64 = base64_encode(sig);
    return encryptedBase64 + "|" + sigB64;
}
```

- Sends as: encryptedBase64 | signatureBase64

4.2.3. Response Authenticity + Confidentiality (Server → Client)

- Client verifies HMAC signature before decrypting.

```
std::string encryptedB64 = signedPayload.substr(0, sep);
std::string signatureB64 = signedPayload.substr(sep + 1);
std::string expectedSignatureB64 = base64_encode(hmac_sha256(g_hmac_key, encryptedB64));

if (!constant_time_equal(signatureB64, expectedSignatureB64)) {
    throw std::runtime_error("Response signature verification failed");
}
```

- Client decrypts response using **RSA OAEP** with its private key (PEM provided via embedded text or local file).

```
std::vector<unsigned char> hashObject(hashObjectSize);
std::vector<unsigned char> hash(hashLen);

if (BCryptCreateHash(hAlg, &hHash, hashObject.data(), hashObjectSize,
    (PUCHAR)key.data(), (ULONG)key.size(), 0) != 0)
    throw std::runtime_error("BCryptCreateHash failed");

BCryptHashData(hHash, (PUCHAR)data.data(), (ULONG)data.size(), 0);
BCryptFinishHash(hHash, hash.data(), (ULONG)hash.size(), 0);

BCryptDestroyHash(hHash);
BCryptCloseAlgorithmProvider(hAlg, 0);

return hash;
```

4.2.4. Bulk Data Encryption (ZIP chunks)

- For large transfers, Malzahar uses **AES-256-GCM** per chunk.
- Malzahar firstly divides the data by 80 mb chunks (cloudflare tunneling used in Nexus site only allows upto 100 mb 80 is chosen to be safe) and uses it's magic per chunk.
- Each chunk is encrypted with:
    - random 12-byte IV
    - 16-byte GCM tag

```
status = BCryptEncrypt(hKey,
    (PUCHAR)plain.data(), (ULONG)plain.size(),
    &authInfo,
    nullptr, 0,
    cipher.data(), (ULONG)cipher.size(), &cbCipher, 0);

BCryptDestroyKey(hKey);
BCryptCloseAlgorithmProvider(hAlg, 0);

if (status != 0) {
    throw std::runtime_error("BCryptEncrypt(GCM) failed");
}
cipher.resize(cbCipher);
```

```
cipher.resize(cbCipher);

return base64_encode(iv) + "|" + base64_encode(cipher)
    + "|" + base64_encode(tag);
```

Payload layout for chunk encryption output:

- IV + Ciphertext + Tag

This means:

- control-plane security: RSA + HMAC
- data-plane security: AES-GCM with retryable chunks

4.3. Request / Response Flow

Most commands use a shared function that creates a signed encrypted payload and sends it via HTTPS POST.

4.3.1. General Flow:

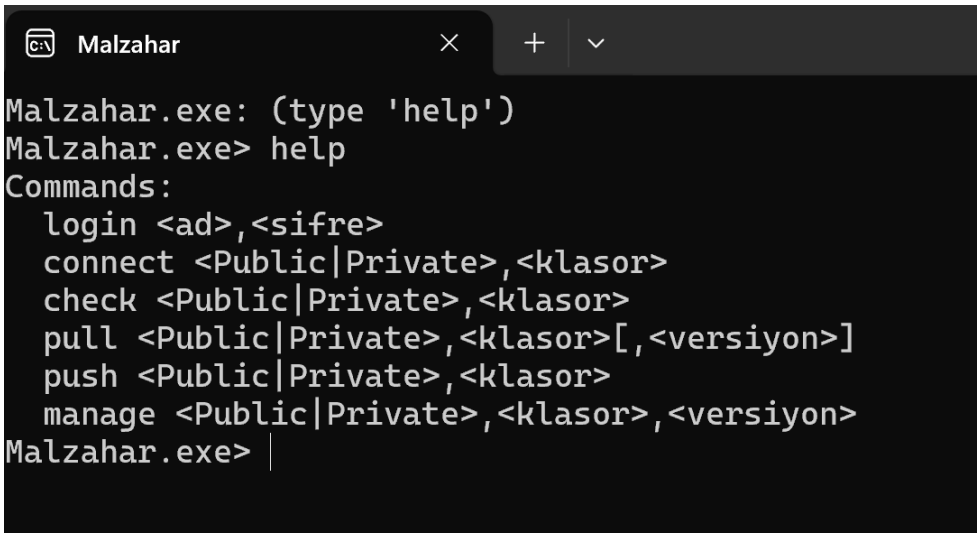
1) Build request JSON (includes request type, credentials, and request data)
2) RSA encrypt JSON with server public key
3) Base64 encode encrypted blob
4) HMAC-SHA256 sign encryptedBase64
5) POST to /api/private/transfer

4.3.2. Response Flow:

1) Split encryptedB64|signatureB64
2) Verify signature with HMAC key
3) RSA decrypt encryptedB64 using client private key (OpenSSL)
4) Parse response JSON

4.4. Command-Line Interface (CLI)

Malzahar is an interactive shell:



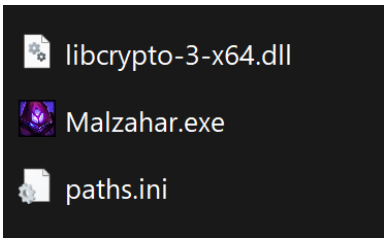
Supported Commands:

- login <name>,<password> / Sends data to server and if server accepts holds the data locally.
- connect <Public|Private>,<folder>,<path> / Matches your paths to server so they are usable.
- check <Public|Private>,<folder> / Shows all the existing versions of a entry.
- pull <Public|Private>,<folder>[,<version>] / Gets the servers version
- push <Public|Private>,<folder> / Uploads your version to server
- manage <Public|Private>,<folder>,<version> / Allows you to delete any version

Important Behavior:

- Almost all operations require login first.
- Folder operations require a mapping to exist in paths.ini (set by connect).

4.5. Folder Mapping



Malzahar maintains a local configuration file called paths.ini. Each entry format is like Scope,Folder-Path. For example; Private,Documents-C:\Users\Me\Documents\ProjectX.
When the user runs connect, Malzahar:

- sends scope/folder to the server
- if successful, it stores the local folder mapping into paths.ini
- later commands (check/pull/push/manage) look up the folder path from this file (case-insensitive match supported)

This design keeps CLI usage fast and avoids repeatedly typing local paths.

4.6. ZIP Handling

Malzahar transfers directories as ZIP archives.

Push:

- Creates a temporary ZIP in the system temp directory.
- Uses PowerShell Compress-Archive.
- If folder is empty, ZIP creation is treated as an error.

```
if (!create_zip_with_powershell(sourcePath, tempZip)) {
    std::cout << "push failed: zip create failed (folder may be empty)\n";
    continue;
}

std::cout << "push: preparing aes key ...\n";
std::vector<unsigned char> aesKey = generate_aes_key_256();
std::string aesKeyB64 = base64_encode(aesKey);

// server'a canonical (ini'deki) scope/klasor gönder
std::string pushData = entry.scope + "/" + entry.klasor + "/" +
    aesKeyB64;
std::string pushControlPayload = build_signed_request_payload("push",
    logined_ad, logined_sifre, pushData);

std::cout << "push: sending control payload ...\n";
std::string controlResp = trim(http_post_raw(TRANSFER_PATH, pushControlPayload));
if (controlResp != "0") {
    std::cout << "push failed: control step rejected, "
        << format_server_response(controlResp)
        << "\n";
    std::error_code ec;
    std::filesystem::remove(tempZip, ec);
    continue;
}
                std::cout << "push: control ok, uploading zip in chunks ...\n";
const size_t CHUNK_SIZE = 80ull * 1024ull * 1024ull; // 80 MiB plaintext chunk
bool ok = push_zip_in_chunks(tempZip, aesKey, CHUNK_SIZE);
```

Pull:

- Downloads ZIP content in chunks into a temp .zip.
- Extracts via PowerShell Expand-Archive into the mapped local folder.
- Clears existing local directory contents before extraction (ensures exact sync style).
- Includes a “flatten single root folder” helper: if the ZIP expands into a single top directory, it moves inner contents up to reduce nesting.

```
bool ok = pull_zip_in_chunks(tempZip, aesKey, totalChunks);
if (!ok) {
    std::error_code ec;
    std::filesystem::remove(tempZip, ec);
    std::cout << "pull failed: chunk download aborted\n";
    continue;
}

bool extracted = extract_zip_with_powershell(tempZip, targetPath);
std::error_code ec;
std::filesystem::remove(tempZip, ec);
```

4.7. Chunked Transfer (Push / Pull)

4.7.1. Push:

Push has two steps:

Step A — Control handshake

- Generates a random AES-256 key.
- Sends scope/folder/aesKeyB64 to the server via signed RSA payload.
- If server returns success (0), the client proceeds.

```
std::filesystem::path sourcePath = entry.path; // canonical path
if (!std::filesystem::exists(sourcePath) || !std::filesystem::is_directory(sourcePath)) {
    std::cout << "push failed: source path is missing or not a directory\n";
    continue;
}

std::cout << "push: creating zip from " << sourcePath.string() << " ...\n";
auto now = std::chrono::steady_clock::now().time_since_epoch().count();
tempZip = std::filesystem::temp_directory_path() /
    ("malzahar_push_" + std::to_string(now) + ".zip");
```

Step B — Chunk upload

- Reads ZIP file in fixed plaintext chunk sizes (e.g., 80 MiB).
- Encrypts each plaintext chunk with AES-GCM.
- Sends to /pushzip as binary [idx (LE u32)][total (LE u32)] + (IV|Cipher|Tag)
- Implements retries 3 retries max per chunk, then aborts upload.

```
std::cout << "push: control ok, uploading zip in chunks ...\n";
const size_t CHUNK_SIZE = 80ull * 1024ull * 1024ull; // 80 MiB plaintext chunk
bool ok = push_zip_in_chunks(tempZip, aesKey, CHUNK_SIZE);

std::error_code ec;
std::filesystem::remove(tempZip, ec);

if (!ok) {
    std::cout << "push failed: chunk upload aborted\n";
    continue;
}

std::cout << "push ok: uploaded successfully\n";
```

4.7.2. Pull:

Pull expects the server to respond (encrypted + signed) with JSON including:

- response: "0"
- aes_key: "<base64>"
- total_chunks: <int>

Then:

- For each chunk index:
  - client POSTs [idx][total] to /zipchunk
  - server returns [idx][total] + IV + ciphertext + TAG
  - client decrypts and appends plaintext to temp zip
- After all chunks complete:
  - extracts ZIP and cleans up temp file
- Also includes retries (3 attempts per chunk)

```
std::cout << "pull: downloading zip in chunks ... total=" << totalChunks << "\n";
bool ok = pull_zip_in_chunks(tempZip, aesKey, totalChunks);
if (!ok) {
    std::error_code ec;
    std::filesystem::remove(tempZip, ec);
    std::cout << "pull failed: chunk download aborted\n";
    continue;
}

bool extracted = extract_zip_with_powershell(tempZip, targetPath);
std::error_code ec;
std::filesystem::remove(tempZip, ec);

if (!extracted) {
    std::cout << "pull failed: zip extraction failed\n";
    continue;
}

std::string rootFolder = flatten_single_root_folder_if_needed(targetPath);
std::string versionInfo = version.empty() ? (rootFolder.empty() ? "latest" : rootFolder) : version;
```

5. Project Analysis

5.1. Test

The project is designed to run on Windows and uses native components (WinHTTP + BCrypt + Crypt32). The transfer mechanism is designed with:

- long timeouts for upload/download (up to ~10 minutes)
- per-chunk retry logic (3 times)
- AES-GCM for both confidentiality and integrity at chunk level

Done tests:

- Login success/failure (wrong password, missing user)
- Push folder with large data (>1 GB) to validate chunking + retry robustness
- Pull large folder and verify byte-for-byte match with ZIP extraction output
- Network interruption mid-transfer and verify retry behavior

5.2. Upgradable Things

Areas that can improve Malzahar significantly:

- **Transport robustness**
  - resumable transfers (save last successful chunk, resume without restarting)
- **Performance**
  - parallel chunk upload/download to increase throughput (multi-threaded chunks)
- **Better structured JSON parsing**
  - current parsing uses regex for simple key-value; a proper JSON library would be safer and cleaner.
- **Better error telemetry**
  - more detailed server responses (and better client-side logging categories)
- **Cross-platform support**
  - currently Windows-specific due to WinHTTP, BCrypt, and PowerShell zip operations.

6. Project Comments

6.1. Possible Usages

- Personal secure file sync between machines (Public/Private scopes)
- "One-command deploy" of project folders to a remote storage controlled by your server
- Lightweight alternative workflow for moving folders without exposing raw HTTP endpoints
- A base infrastructure for a future GUI-based sync client

6.2. To-Do's When Using

- Ensure hmac.key and private_key.pem exist (unless embedded in code).
- Run connect first to bind a local path into paths.ini.
- Avoid using extremely large chunk sizes on unstable networks (smaller chunks can reduce retransfer cost).
- Keep PowerShell available in the environment (required for zip operations).

6.3. Creator Comments

The project has gone beyond my expectations. I thought it would be only a simple file transfer structure. But implementation of RSA and AES intrigued me. It was really fun to work on and awesome to pull this kinda project alone.

-Hooah, Famura

7. Credits

- Famura Only Project

8. References

- NIST. (2001). *Advanced Encryption Standard (AES)*. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
- NIST. (2015). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*. National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
- Rivest, R., Shamir, A., & Adleman, L. (1978). *A method for obtaining digital signatures and public-key cryptosystems*. Communications of the ACM, 21(2), 120–126. https://doi.org/10.1145/359340.359342
- Rescorla, E. (2000). *HTTP Over TLS*. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc2818
- Microsoft. (2024). *WinHTTP API documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/winhttp/about-winhttp
- Microsoft. (2024). *Cryptography API: Next Generation (CNG)*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/seccng/cng-portal
- Microsoft. (2024). *Crypt32 API documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptography-functions
- OpenSSL Project. (2024). *OpenSSL Cryptography and SSL/TLS Toolkit*. https://www.openssl.org/docs/
- Microsoft. (2024). *PowerShell Compress-Archive*. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.archive/compress-archive
- Microsoft. (2024). *PowerShell Expand-Archive*. Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.archive/expand-archive

9. Disk Contents

- Source code of project
- This report on .pdf format
- This reports on .docx format
- Build of this project
- (I Just) Died In Your Arms by Cutting Crew